

Computer Vision (Assignment 3)

Stereo Vision

Ahmad Yasser Mohammad Salaheldin Hamad (ID:9074)

Moamen Talaat (ID:9103)

Table of Contents

1	Introduction	2
2	Implementation Details	2
2.1	Cost Metrics	2
2.2	Experimental Parameters	2
3	Results and Discussion	3
3.1	Observation on Window Size (w)	4
3.2	Observation on Metrics (SAD vs. SSD)	4
4	Conclusion	4
	Part 2 (Dynamic Programming)	5
	Code Explanation	5

1 Introduction

The objective of this assignment, as specified was to implement a simple dense stereo depth estimation algorithm: Block Matching. Using a forward-facing camera setup translated along the horizontal axis, we compute the horizontal disparity for pixels along each scanline. Disparity is inversely proportional to depth; objects with high disparity (large horizontal shift) are interpreted as being closer to the camera.

2 Implementation Details

As required, the solution was implemented in Python using numpy for numerical array operations and cv2 (OpenCV) for basic image I/O.

2.1 Cost Metrics

We implemented two mathematical metrics for measuring window similarity:

1. **SAD (Sum of Absolute Differences):** A computationally fast metric that sums the absolute pixel-wise differences between windows.

$$\text{Cost}_{\text{SAD}} = \sum_{(i,j) \in W} |I_l(i,j) - I_r(i,j - d)| \quad (1)$$

2. **SSD (Sum of Squared Differences):** A metric more sensitive to large outliers due to the squaring operation.

$$\text{Cost}_{\text{SSD}} = \sum_{(i,j) \in W} (I_l(i,j) - I_r(i,j - d))^2 \quad (2)$$

Here, W is the matching window of size w , and d is the disparity shift being evaluated.

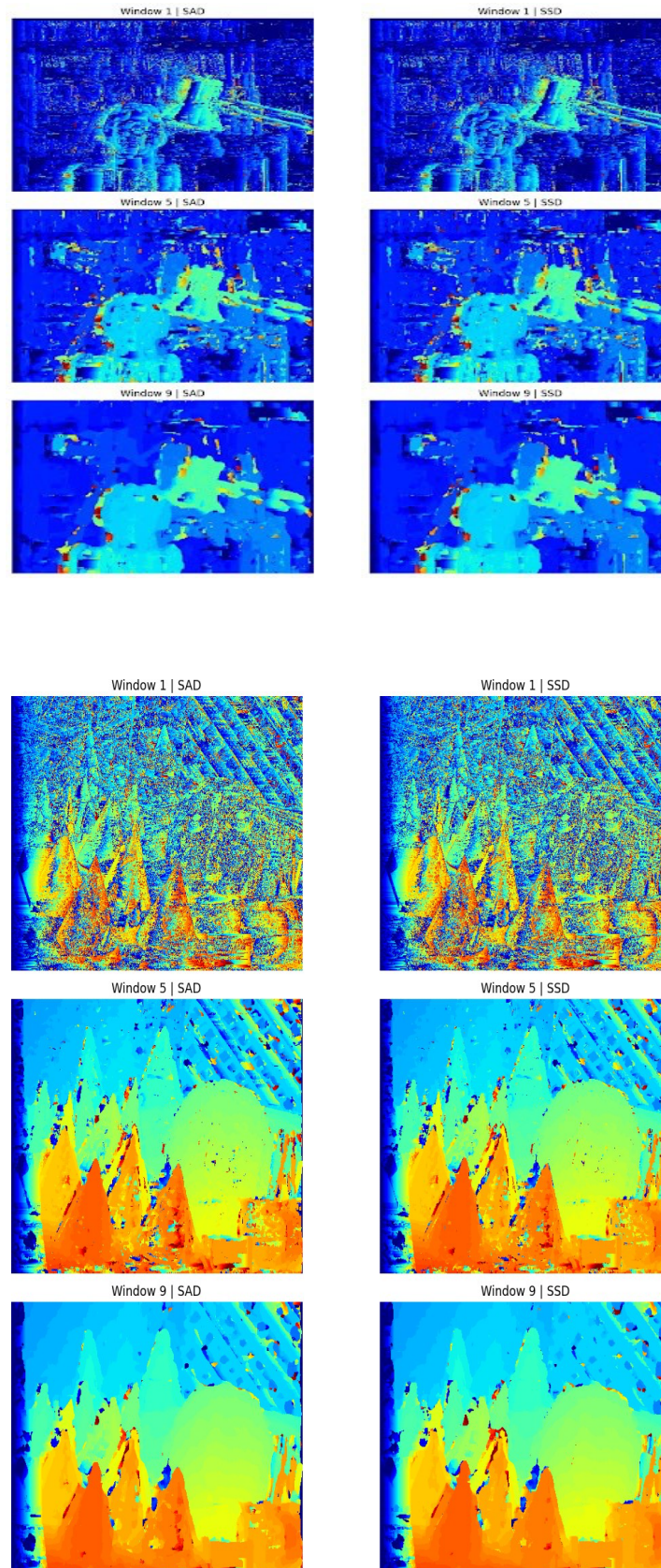
2.2 Experimental Parameters

We tested the block matching algorithm across six scenarios:

- **Window Sizes:** $w = 1, 5, 9$.
- **Metrics:** SAD and SSD for each window size.
- **Maximum Disparity:** A maximum horizontal search range of 64 pixels was utilized.

3 Results and Discussion

The generated output maps, visualized using the 'jet' colormap, are presented in Figure 1. Warmer colors (reds, yellows) represent higher disparity (near objects), while cooler colors (blues) represent lower disparity (far background).



3.1 Observation on Window Size (w)

$w = 1$ (Top Row): These maps show significant salt-and-pepper noise. A single pixel is highly sensitive to sensor noise, leading to multiple incorrect best-cost matches.

$w = 5$ (Middle Row): Noise is significantly reduced, and the relative depth of the foreground figure becomes coherent.

$w = 9$ (Bottom Row): The maps are the smoothest and clearest. However, a common artifact of large windows—object boundary "fattening" or blurring is observed, particularly around the edges of the main figure. There is a trade off between noise reduction and edge precision.

3.2 Observation on Metrics (SAD vs. SSD)

The results for SAD (left column) and SSD (right column) are visually nearly identical for this dataset. This is likely due to consistent lighting between the left and right images. While SSD is theoretically more sensitive to large outliers (noise), its results here did not differ substantially from the simpler SAD metric.

4 Conclusion

In this part of the assignment, we successfully implemented and visualized simple block-based stereo vision. The experiments highlighted the fundamental trade-off of window selection: larger windows reduce pixel noise but degrade depth boundary precision. Both SAD and SSD provided effective results for basic horizontal disparity estimation.

Part 2 (Dynamic Programming)

The objective of this section is to be able to develop a disparity map for images. A disparity map measures pixel movement in a 2D space. It shows how many pixels a specific point moved horizontally between two images. The values in the map are measured through pixels (or the difference in them). Disparity maps are an initial step performed in preparation for depth estimating the depth of objects in given images.

Code Explanation

The code is mostly done in a single huge block, doing all necessary mathematical procedures at once. Currently, three images are present. For every image, we have the left and right version of the image. We are allowed to assume that the images are taken from parallel cameras (i.e. the epipoles are at infinity). The cameras are not converging. This assumption means that neither image would have to be reprojected through a homography transformation, thus further simplifying the transformations necessary for this task. Accordingly, we can now visualize that both versions of the images are taken with two cameras (or one) in which there was a sufficiently enough distance between both cameras. Accordingly, this reduces the task of matching a point in one image (typically a point with representative features such as an edge for example) from moving across the entire second image and trying to correspond pixels from the point to each and every pixel in the image to a simple scan across a horizontal line. It must also be noted that due to the aforementioned parallel cameras assumption, we are now only varying the x axis, not both the x and y axes, which, once again, simplifies the procedure undergone in this code.

The code initially opens the grayscale version of the left and right images. It then loops through every row of the image. It then processes every pixel from the left image and compares it to all pixels of the right image at the same row. It computes a cost function between each correspondence according to the following cost formula:

$$d_{ij} = \frac{(I_l(i) - I_r(j))^2}{\sigma^2}$$

Where σ is the measure of pixel noise (and is considered at a fixed value of 2). Also, $I_l(i)$ and $I_r(j)$ are the left and right scanlines, respectively. Pixels in each scanline are either matched or skipped. The cost of skipping a pixel in either scanline is a constant c_o of value 1. Moreover, for initialization, a

matrix D is initialized with zeros in the beginning of each row. This matrix stores intermediate values and is crucial for future backpropagation steps. $D(1,1) = d_{ij}$ and

$$D(i,j) = \min (D(i-1,j-1) + d_{ij}, D(i-1,j) + c_o, D(i,j-1) + c_o)$$

The aforementioned equation serves as the dynamic programming recursive relation. For the subsequent backtracking section, the code looks for which minimum value was recorded in each i and j . If it was $D(i-1,j-1) + d_{ij}$, the outcome matrix stores the absolute value of the difference between i and j . If it was $D(i-1,j) + c_o$, this corresponds to skipping a pixel in the left image, so the left disparity matrix is populated with 0. Finally, if it was $D(i,j-1) + c_o$, this corresponds to skipping a pixel in the right image, so the right disparity matrix is populated with 0.

Below are some outcome figures:

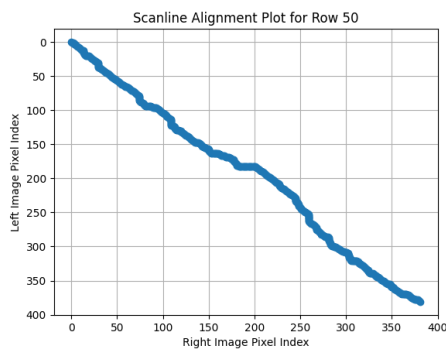


Figure 1 Scanline alignment for the first image at row 50

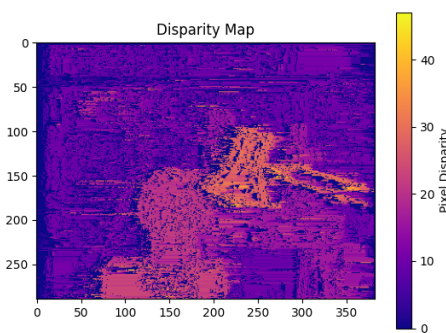


Figure 2 Left disparity map for first image at row 50

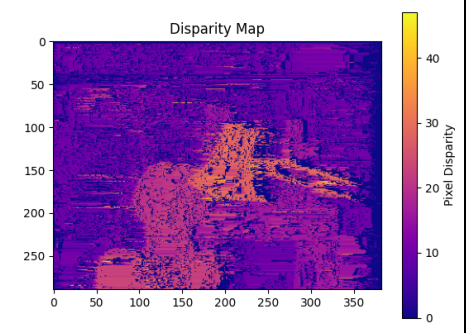


Figure 3 Right disparity map for first image

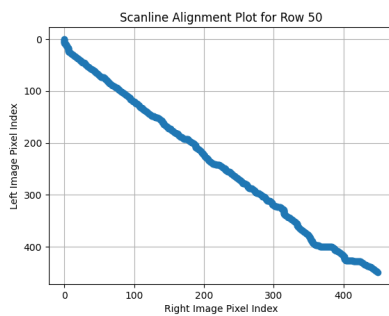


Figure 4 Scanline alignment for the second image at row 50

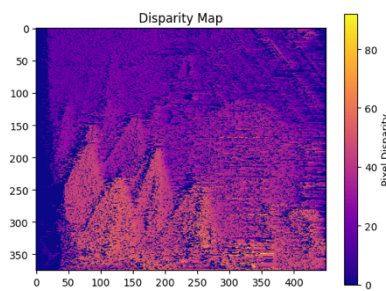


Figure 5 Left disparity map for second image

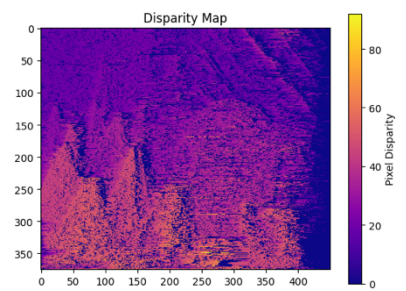


Figure 6 Right disparity map for second image

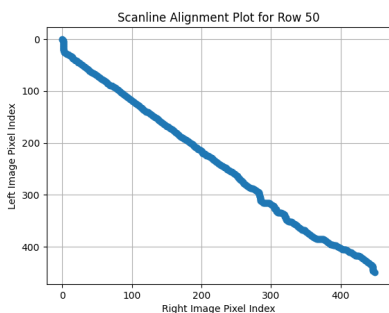


Figure 7 Scanline alignment for the third image at row 50

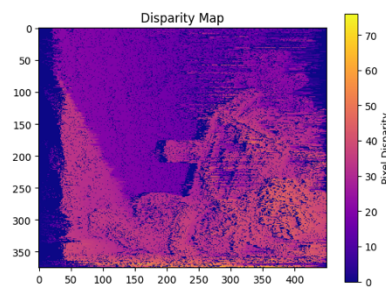


Figure 8 Left disparity map for third image

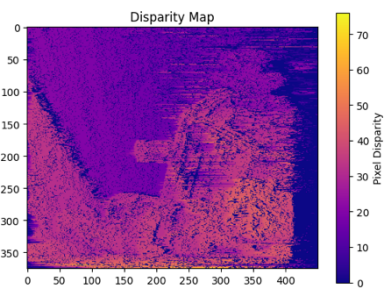


Figure 9 Right disparity map for third image